Digital Signal Processing – Assignment 2
FIR Filters, Adaptive LMS Filtering, and Realtime ECG Peak Detection

Saad Aziz (GUID: 2917444A)
William Chambers (GUID: 2906463C)

University of Glasgow, School of Engineering

November 12, 2026

Recordings

Two ECGs were recorded in the lab following the Moodle instructions, with one electrode on the wrist and one on the ankle; the wrist electrode was the ground. `ecg1.tsv` is Saad's ECG and is the recording analysed in Tasks 1–5; it was chosen because a significant baseline wander was present. `ecg2.tsv` is Will's ECG and is used only for the matched-filter template in Task 5. The ECG is in column 8 of each file (counting from 0). The data was sampled at 250 Hz, so the Nyquist frequency is 125 Hz.

Task 1: Time and Frequency Analysis

The spectrum is $|\mathrm{FFT}(x)|/N$, plotted on linear axes. Only the bins up to $f_s/2$ are shown, because the spectrum of a real signal is mirrored. Figures 1 and 2 show the raw recording in the time and frequency domains.

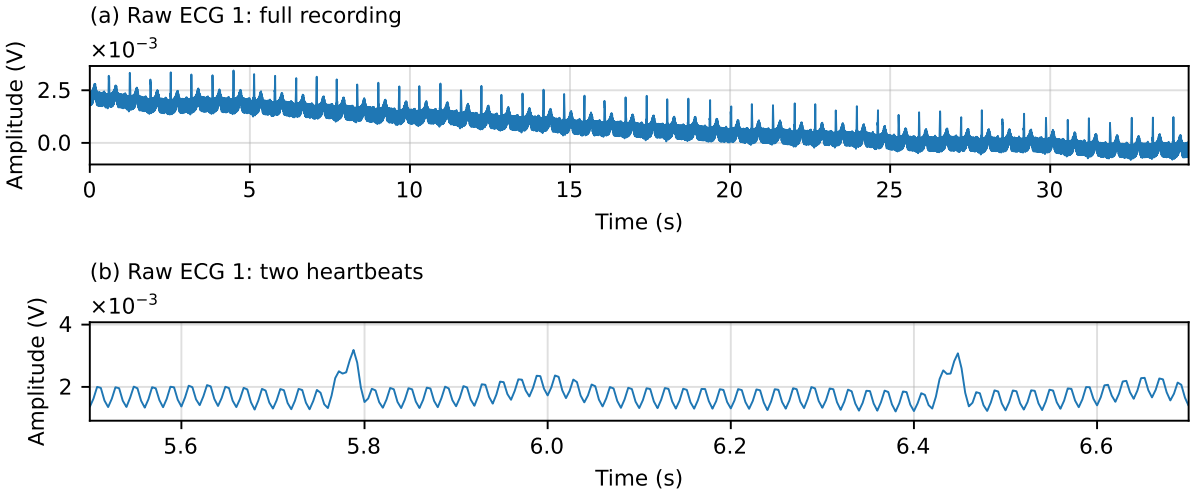


Figure 1: Raw ECG (`ecg1.tsv`) in the time domain: (a) the full recording, with the baseline drifting downwards; (b) two heartbeats, with the 50 Hz ripple.

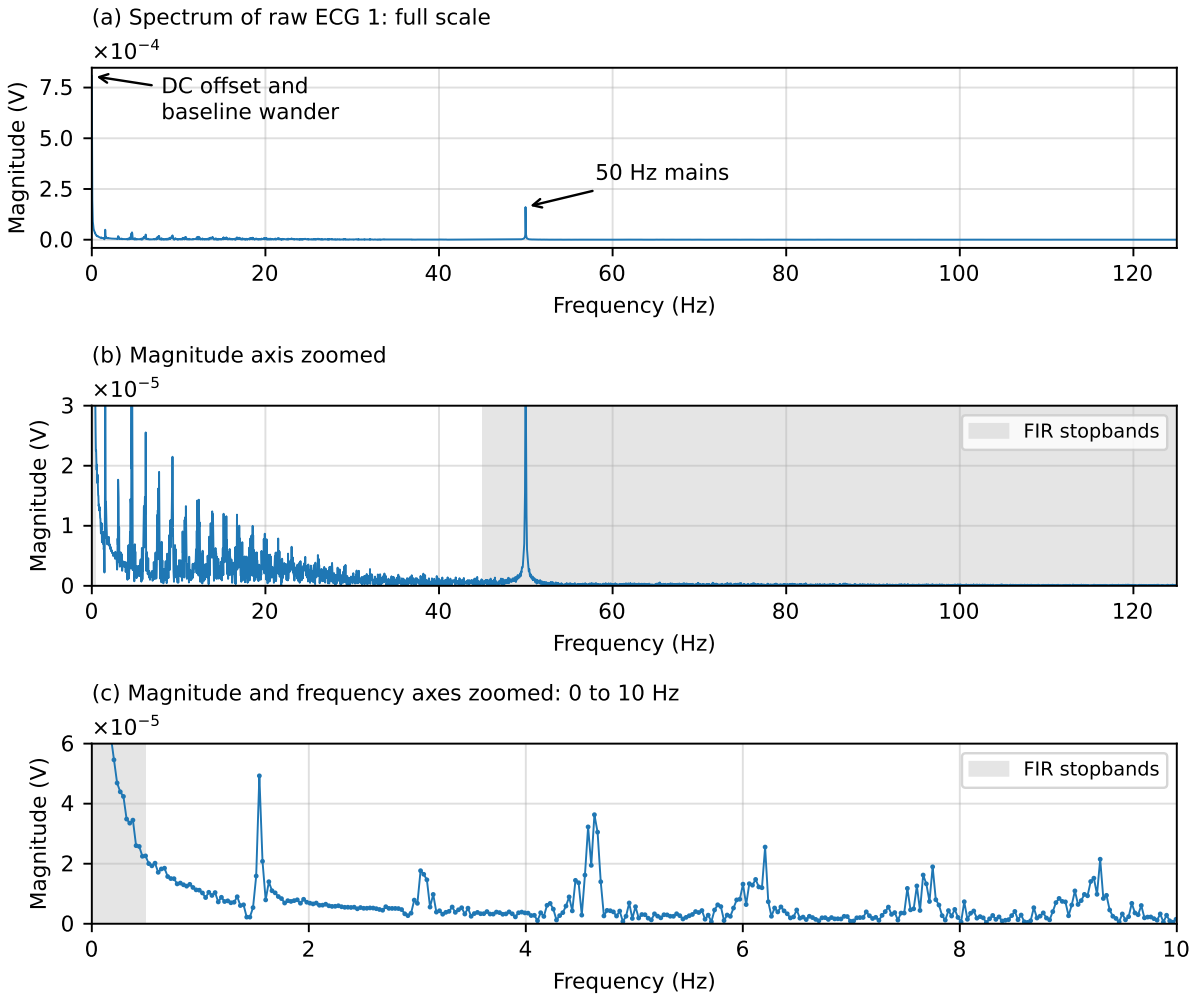


Figure 2: Spectrum of the raw ECG on linear axes: (a) full scale, dominated by the spike at 0 Hz and the 50 Hz peak; (b) magnitude axis zoomed to show the ECG harmonics; (c) the lowest 10 Hz, with the heart-rate fundamental and its harmonics. The shaded regions are the two stopbands chosen for the FIR filter.

ECG signal and harmonics. In Figure 1(b) the two R peaks are about 0.66 s apart, which is a heart rate of about 1.5 Hz. Over the whole recording the spectrum has its first peak at about 1.55 Hz (Figure 2(c)), followed by a series of evenly spaced peaks at its multiples, at about 3.1, 4.6 and 6.2 Hz. Because they sit at multiples of the heart rate, they are the harmonics of the heartbeat and not noise. The series fades out by about 40 Hz (Figure 2(b)).

DC offset and baseline wander. The trace starts about 2 mV above zero and drifts downwards over the recording (Figure 1(a)). The R peaks keep the same base-to-peak amplitude throughout the recording, so the drift is a simple offset to the ECG trace. In the spectrum it is the large spike at 0 Hz and the bins just above it.

50 Hz mains. Visible as the regular ripple on the trace in Figure 1(b) and as the sharp peak at 50 Hz in the spectrum. It sits at exactly the UK mains frequency, so it is mains hum picked up by the body and the leads. Mains hum usually comes with harmonics, but there is no peak at 100 Hz.

Muscle noise (EMG). Muscle activity adds broadband noise, which may be part of the low noise floor between the ECG harmonics in Figure 2(b). This is broadband and overlaps with the ECG harmonics, so it cannot be removed with a frequency-selective filter without affecting the heartbeat itself.

Frequencies to remove. Two ranges are removed. 0 – 0.5 Hz removes the DC offset and the baseline wander, and stays well below the 1.5 Hz fundamental of the heartbeat. 45 – 125 Hz removes the 50 Hz mains and everything above it up to Nyquist, where the spectrum contains no visible ECG harmonics. The edge is at 45 Hz because the ECG harmonics have faded by about 40 Hz and the foot of the 50 Hz peak starts a few hertz below it, so 45 Hz lies in the gap between the two. It also leaves room for the transition band of the filter, which is caused by the window function (Task 2) and has to end before 50 Hz.

Task 2: FIR Filter Coefficient Design

Number of coefficients. `fir_design(sample_rate, ranges)` takes the sampling rate and a 2D array of stopbands, each given as [low, high] in hertz. The number of coefficients, M, is calculated from the narrowest stopband passed to the function, here 0 – 0.5 Hz. The frequency bins of the design are spaced at half this width, 0.25 Hz, so that even the narrowest stopband always has at least two bins to act on. The number of coefficients is then the sampling rate divided by the bin spacing, $250/0.25 = 1000$. It is made even so that the impulse response can be split into two equal halves when it is shifted (see below).

The cost of a fine resolution is delay. The centre of the impulse response is at tap $M/2$, so the output is delayed by $M/(2f_s) = 2.0$ s. `fir_realtime_sim.py` prints 1.998 s because it uses $(M-1)/2$; after the half swap our centre is at tap $M/2$, so the true delay is 2.000 s. The difference is half a sample and does not show in any of the plots. The delay depends only on the bin spacing, which was set above by the narrowest stopband: a filter that cuts closer to DC needs finer bins and so a longer delay; halving the spacing doubles the delay for a constant sampling rate.

IFFT design and edge cases. The bin frequencies kf_s/M run from 0 to just below f_s. They are folded about $f_s/2$, so that bin $M-k$ gets the same frequency as bin k. The ideal response H starts as all ones (all-pass), and every bin whose folded frequency lies inside one of the stopbands is set to zero. Because both halves of the spectrum go through the same test, H is mirrored, which makes the IFFT real (`np.real` only removes rounding residue). There are two edge cases: bin 0 (DC) and bin $M/2$ (Nyquist). Neither has a partner, because the mirror of bin 0 would be index M, which is outside the array, and bin $M/2$ is its own mirror. Zeroing each stopband and then its mirror $M-k$ by hand would need a special case for both. Our design never looks up a mirror: every bin is tested once against its own folded frequency. So bin 0 is removed by any band starting at 0 Hz and bin $M/2$ by any band ending at $f_s/2$, with no extra code.

The IFFT returns an impulse response. Swapping the two halves moves the centre to tap $M/2$. This makes the filter causal and keeps the coefficients symmetric about the centre tap, so the filter has linear phase. Thus, group delay is constant and the shape of the heartbeat is preserved. Finally the coefficients are multiplied by a Hamming window function, which tapers the truncated ends and reduces the ripple at the edges of the bands (Gibbs phenomenon).

Figure 3 shows the result. The response does not switch instantly: the edges at 0.5 and 45 Hz lie in the middle of a transition band roughly 0.5 Hz wide (Figures 3(c) and (d)), which is the price of the window. This is also why 1000 coefficients are enough. The gain is back to 1 by

about 1 Hz (Figure 3(c)), below the 1.5 Hz fundamental of the heartbeat, and the upper edge has closed by about 46 Hz (Figure 3(d)), before the 50 Hz mains.

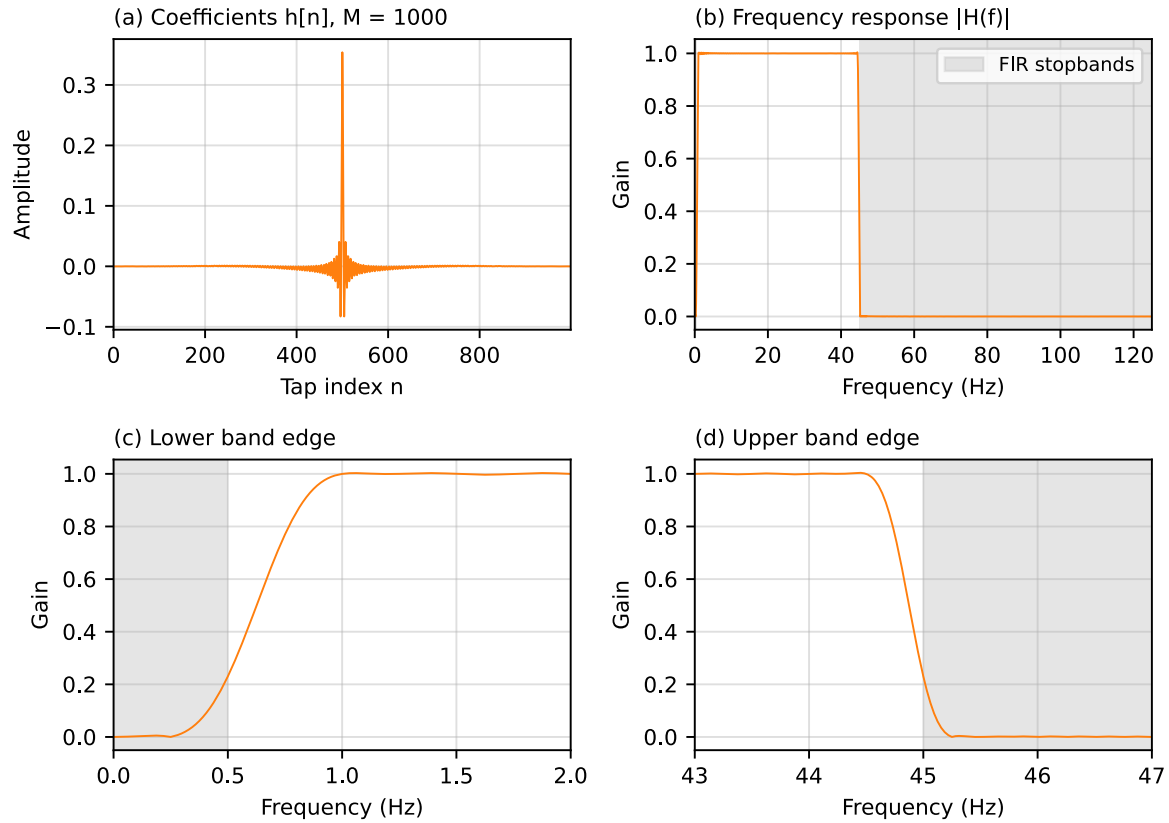


Figure 3: The designed FIR filter ($M = 1000$): (a) coefficients after the half swap and the Hamming window, centred on tap $M/2 = 500$; (b) frequency response, with the stopbands shaded; (c) and (d) the two band edges, which lie inside the transition bands.

Task 3: Realtime FIR Filtering

Ring buffer. `FIRfilter` stores the coefficients, a buffer of the same length filled with zeros, and `offset`, the index at which the next sample is written. `dofilter(v)` takes one sample and returns one sample, $y[n]$. The new sample overwrites the oldest one at `offset`, and then only `offset` moves on, wrapping back to 0 at the end of the buffer. Reading backwards from `offset` gives the samples from newest to oldest, but the read wraps round the end of the array, so the sum is calculated as two dot products.

Filtered ECG. `run_fir_filter` in `fir_realtime_sim.py` feeds the ECG into `dofilter` one sample at a time. The result is shown in Figure 4.

Start-up. The buffer starts as zeros, so for the first $M = 1000$ samples (4 s) the filter sees a step from zero up to the DC offset of the recording and its output is not yet valid. The largest swing comes after about 2 s, when the step reaches the centre tap. Once the buffer is full the baseline sits on 0 V and the drift is gone (Figure 4(a)).

PQRST. In Figure 4(b) the filtered trace is shifted back by the 2.0 s delay so that it lies on top of the raw trace. The 50 Hz ripple is gone, and the P wave, the QRS complex and the T wave keep their shape and timing, so the heartbeat is not distorted.

Spectrum. In Figure 4(c) the spike at 0 Hz and the 50 Hz peak are gone, while the ECG harmonics between the two stopbands are unchanged. Components close to 0.5 Hz and 45 Hz lie in the transition bands (Figure 3), so they are attenuated but not removed.

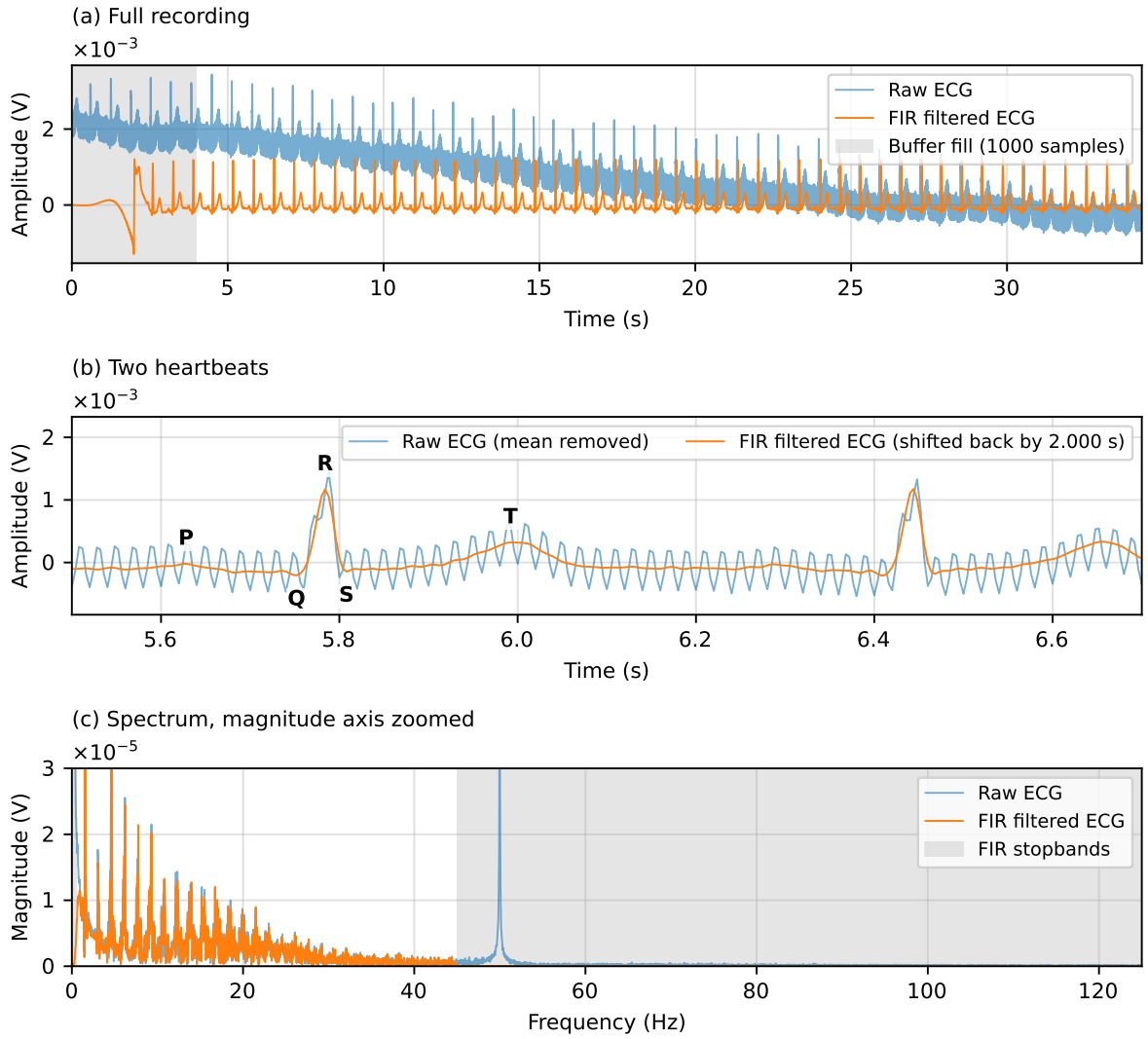


Figure 4: ECG filtered sample by sample with the FIR filter: (a) full recording, with the start-up transient while the buffer fills; (b) two heartbeats, with the filtered trace shifted back by the filter delay and P, Q, R, S and T labelled; (c) spectrum before and after filtering.

Task 4: Adaptive LMS Filter

Implementation. `doFilterAdaptive` is a second method of the same class, taking one sample of the signal, one sample of the noise reference and the learning rate. The delay line now holds the noise reference instead of the ECG. `dofilter(noise)` returns $y[n]$, the filter's current estimate of the interference, and the output is the error $e[n] = \text{signal} - y[n]$. Every weight is then adjusted using the same two slices of the ring buffer as `dofilter`, so again nothing is shifted.

Two cancellers in series. `lms_filter.py` uses two of these filters, one after the other, both starting with all weights at zero. The first has one tap and a constant reference of 1; its single weight follows the slowly changing offset, which removes the DC and the baseline wander. The second has 10 taps and the reference $\sin(2\pi \cdot 50\, n/f_s)$ ($\mu = 0.01$), and removes the mains.

Cancelling a sine needs at least two taps, so that both its amplitude and its phase can be matched; ten taps span two full periods of the reference, so the reference power held in the delay line is the same at every sample and the filter adapts at a steady rate. Its learning rate, $\mu = 0.01$, was chosen by trial. A larger μ adapts faster, but the weights start to react to the ECG as well; a smaller μ is smoother, but lets more of the mains through while it adapts. The small residual at 50 Hz in Figure 5(d) shows that a little of the mains still gets through.

The first learning rate was chosen to match the FIR filter. With one tap and a reference of 1 the canceller is a first-order high-pass filter. For small μ its cut-off frequency is about $\mu f_s/(2\pi)$, so $\mu = 0.0125$ gives about 0.5 Hz, the same edge as the lower stopband of the FIR filter.

A single 10-tap filter with the combined reference $1 + \sin(2\pi \cdot 50\, n/f_s)$ was tried first. It removed the mains, but it flattened the slow parts of the heartbeat, most visibly the T wave. The reason is that all ten taps see the constant, so the DC part adapts ten times faster and its cut-off moves from about 0.5 Hz to about 4 Hz, which is above the heart rate. Splitting the filter in two lets the baseline canceller be slow while the mains canceller stays fast.

Comparison with the FIR filter. Figure 5 compares the two outputs.

Shape. With the FIR output shifted back by its delay, the two traces lie almost on top of each other and P, QRS and T are intact in both (Figure 5(c)). The LMS trace sits slightly lower after each QRS complex and T wave, because its first-order high-pass also reacts a little to the heartbeat itself. Unlike the FIR filter, this high-pass does not have linear phase, so the slow parts of the heartbeat are delayed by different amounts and the shape changes slightly. The FIR filter avoids this with its symmetric coefficients (Task 2).

Delay and start-up. The LMS output has no group delay and is plotted without any shift, against 2.0 s for the FIR filter. Its baseline settles within about 1 s, while the FIR filter needs its 4 s buffer fill (Figure 5(b)).

Size and latency. The LMS filter uses $1 + 10 = 11$ weights against 1000 coefficients. Out of curiosity we measured the per sample latency for each filter; this showed that the LMS filter latency is approximately $3\times$ that of the FIR filter. We suppose this is because it has two sequential filters and because it is calling more functions resulting in a higher overhead.

Spectrum. The LMS filter only removes what its references describe, DC and 50 Hz. It keeps everything else, including the content above 45 Hz that the FIR filter removes, and it leaves a small residual at 50 Hz (Figure 5(d)). The two filters therefore do not do quite the same job: the LMS filter needs to know the interference in advance, and the FIR filter removes whole bands whatever they contain.

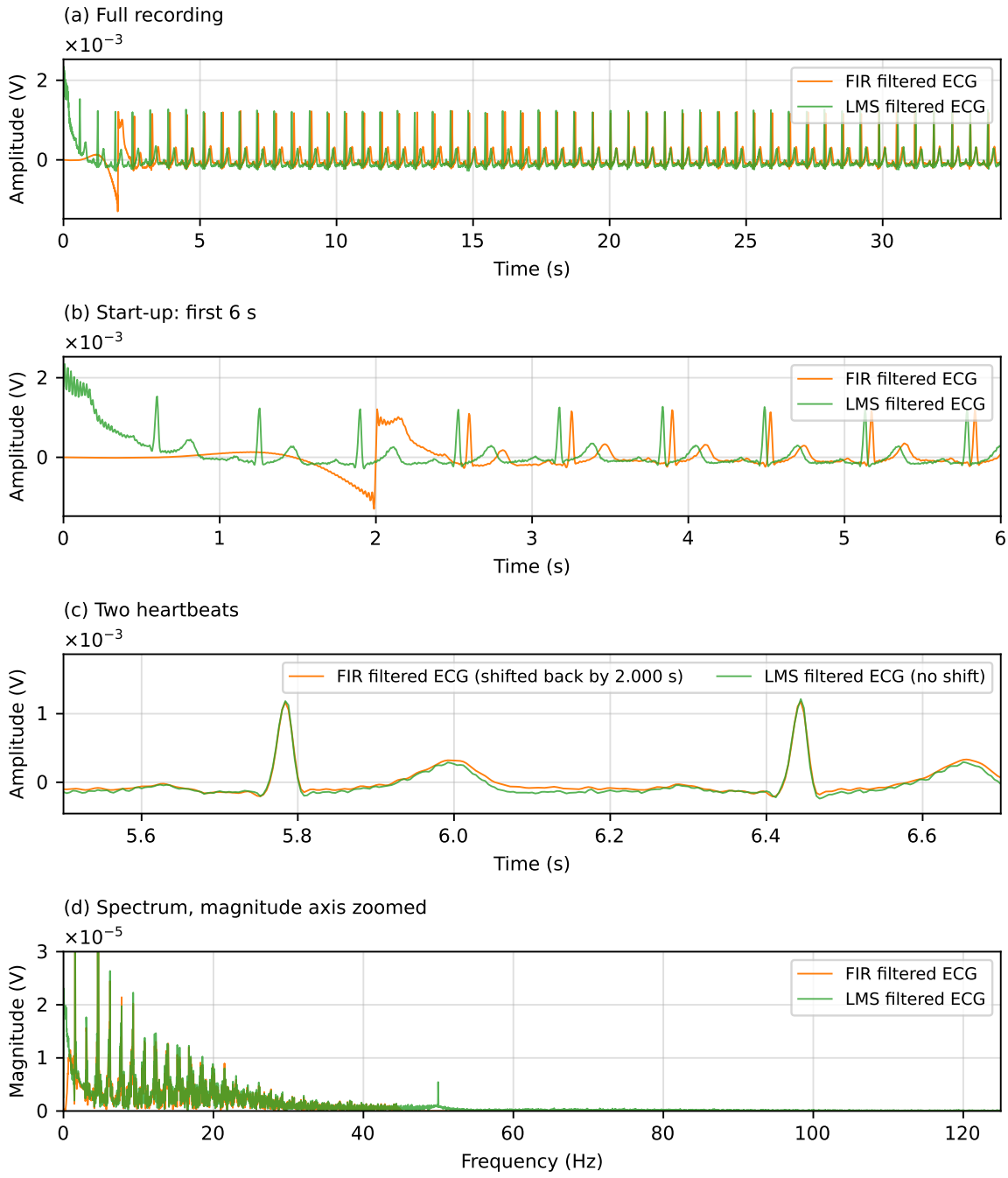


Figure 5: FIR filter (1000 coefficients) against the two LMS cancellers in series (DC: 1 tap, $\mu = 0.0125$; 50 Hz: 10 taps, $\mu = 0.01$): (a) full recording; (b) start-up; (c) two heartbeats, with only the FIR output shifted back by its delay; (d) spectra.

Task 5: ECG Heartbeat Detection

Template and matched filter. `ecg2.tsv` is first cleaned with the same FIR filter as `ecg1.tsv`, sample by sample, so that both the template and the signal have been through the same filter. The template is a 25-sample snippet containing a single QRS peak (Figure 6(a)), centred on R at sample 2576. This beat was chosen late enough in the recording for the cleaning filter's buffer to have filled.

The matched filter is an `FIRfilter` whose coefficients are the template reversed in time. An

FIR filter performs a convolution, which flips its coefficients against the signal, so reversing the template first turns it into a correlation with the template. The output is squared, which makes it positive and makes the beats stand out further from everything else; its unit is therefore V^4 (the filter multiplies the template in volts by the signal in volts, and the result is squared).

Detection. Each sample of the ECG goes through the cleaning filter, the matched filter, the squaring and the tests below before the next sample is read.

Warm-up. The output is ignored for the first $\mathrm{len}(h) + \mathrm{len}(\mathrm{template}) = 1025$ samples, until both buffers hold only real data. The start-up transient of the cleaning filter produces the largest spike of the whole output, at about 2 s. It sits inside this window (Figure 6(b)), so it is ignored; otherwise it would register as a beat.

Threshold. $1.5 \times 10^{-11}\,\mathrm{V}^4$, chosen from the plot. Every beat peaks above about $2.1 \times 10^{-11}\,\mathrm{V}^4$, and the smaller side spike just before each beat stays below about $0.9 \times 10^{-11}\,\mathrm{V}^4$, so the threshold lies in the gap with a margin on both sides (Figure 6(c)). The threshold is fixed and was chosen on the same recording it is used on. The squared output grows with the square of the ECG amplitude, so beats would start to be missed if the ECG got about 15 % smaller ($\sqrt{1.5/2.1} \approx 0.85$), and the side spikes would be counted if it got about 30 % bigger ($\sqrt{1.5/0.9} \approx 1.3$), for example with worse electrode contact or a different person. An adaptive threshold, such as a fixed fraction of the recent peak heights, would follow these changes, at the cost of more code and its own start-up period.

Crossing. A beat is registered at the sample where the output rises through the threshold, so each beat is counted once. This is the upward crossing and not the top of the peak: the crosses in Figure 6 mark the first sample above the threshold, which is sometimes a couple of samples before the peak.

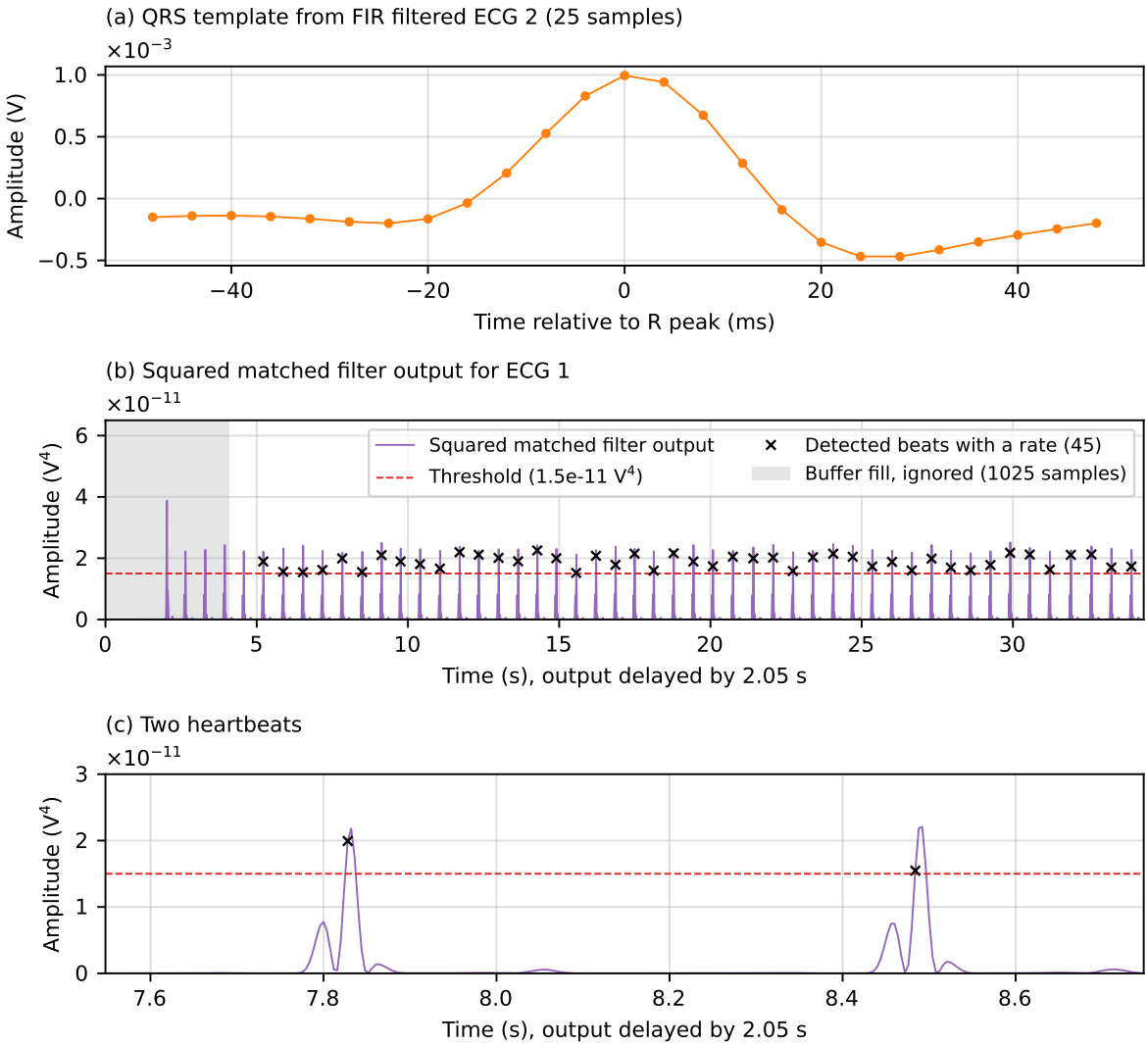


Figure 6: Matched-filter detection: (a) the QRS template cut from the cleaned `ecg2.tsv`; (b) squared matched-filter output for `ecg1.tsv`, with the warm-up ignored, the threshold and the 45 beats that the heart rate is calculated from; (c) two beats, showing the threshold between the beat peaks and the side spikes.

Heuristics. The momentary heart rate is $60\, f_s/(n_{\text{beat}} - n_{\text{last}})$ in beats per minute, where n is the sample index of a crossing. A rate is accepted only if it passes two rules. First, it must lie between 30 and 200 bpm, limits for a human heart suggested by a physiology student; they are also realistic, since a resting heart rarely drops below 30 bpm and a heart at full effort rarely goes above 200 bpm. Second, it must be within ±20% of the previous accepted rate. A missed beat halves the rate. A false detection between two beats splits one interval into two, and at least one of the two rates is then at least doubled. Both fall far outside ±20%, while the rate of this recording changes by only a few bpm from one beat to the next.

Neither heuristic rejected a rate on this recording, so they act purely as safeguards. The second rule has a limitation: it compares with the last *accepted* rate, so a genuine jump of more than 20% from one beat to the next would be rejected, and so would every beat after it.

Momentary heart rate. After the warm-up the detector finds 46 beats. The first has no beat before it and gives no rate, which leaves the 45 rates printed by the script, with a mean of 92.0 bpm (Figure 7). The rate varies between about 86 and 96 bpm. Some of this is expected to

be natural, possibly the heart rate rising and falling with breathing. Some of it is resolution: an interval is a whole number of samples (4 ms each), and at 92 bpm one sample changes the rate by about 0.6 bpm, which shows as the repeated levels in the plot.

The cleaning filter and the matched filter together delay every beat by the same $500 + 12$ samples (2.05 s). A constant delay cancels in the intervals, so the heart rates are not affected by it; the time axis of Figure 7 is shifted back by this delay so that it lines up with the ECG.

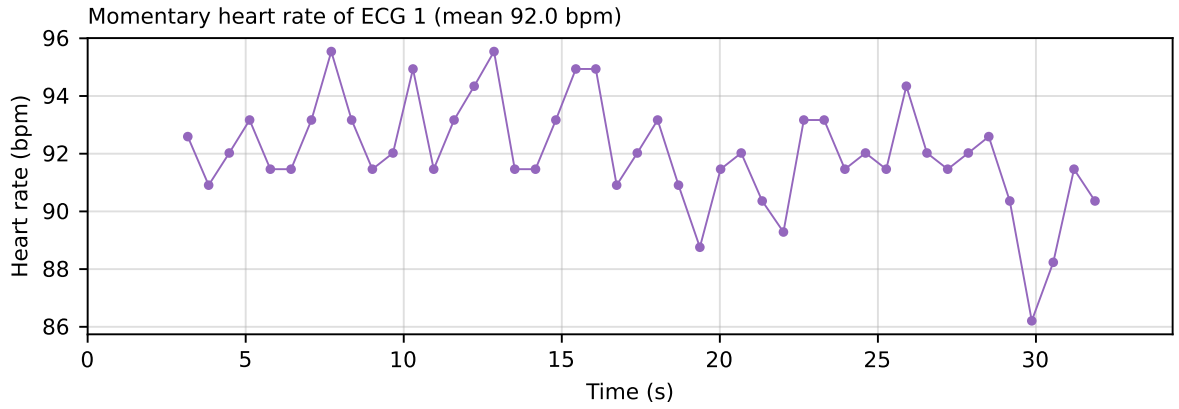


Figure 7: Momentary heart rate of `ecg1.tsv` from the 45 beat-to-beat intervals (mean 92.0 bpm), with the time axis shifted back by the filter delay.

A Acknowledgement of GenAI Use

Tools: Claude Opus 5.5 (Anthropic); Gemini 4 Argon (Google); Gemini 3.1 Pro (Google); Gemini 3 Flash-lite (Google); GitHub Copilot (Microsoft).
Use: Claude Opus 5.5 reviewed and debugged our code; explained theoretical concepts, wrote the plotting code in `heartbeat.py`; suggested optimisation strategies, wrote the script that exports the figures of this report in one style, and resolved Git merge conflicts. Gemini 4 Argon and Gemini 3.1 Pro were used in an almost identical way to Claude Opus 5.5; for refining plots, to generate the LaTeX framework for the report, checking for inconsistencies, linking figures, and for typesetting. Gemini was used for general lookups and GitHub Copilot for inline autocomplete.

B Usage Instructions

From a terminal, while in the project folder, run:

```
python3 ecg_plot.py
python3 fir_realtime_sim.py
python3 lms_filter.py
python3 heartbeat.py
```

The scripts need `numpy`, `scipy` and `matplotlib`, and `ecg1.tsv` and `ecg2.tsv` must be in the same folder. `fir_design.py`, `fir_filter.py` and `support_funcs.py` are imported by the scripts. Each script opens its plots with `plt.show()`. `fir_realtime_sim.py` and `lms_filter.py` print the number of coefficients and the group delay, and `heartbeat.py` prints the number of heart rates and their mean. The figures in this report were drawn from the variables of these scripts by a separate plotting script, so that they share one style; the scripts themselves show the same data.

C Python Code

C.1 support_funcs.py

```python
from pathlib import Path
import numpy as np
import matplotlib.pyplot as plt

BASE_DIR = Path(__file__).resolve().parent


def import_ecg_data(filename):
    """
    Parameters:
    filename: str or Path
        The name of the file containing the ECG data.

    Returns:
    tuple[np.ndarray, np.ndarray, float]
        (t, ecg, sample_rate) - time vector (s), ECG signal (V), and sampling rate (Hz).
    """
    ECG_COLUMN = 8  # ADC channel 2
    data = np.loadtxt(BASE_DIR / filename)
    t = data[:, 0]
    ecg = data[:, ECG_COLUMN]
    sample_rate = 1 / (t[1] - t[0])  # fs = 250 Hz

    return t, ecg, sample_rate
```

C.2 ecg_plot.py

```python
# Task 1

import numpy as np
from scipy.fft import fft
import matplotlib.pyplot as plt

from support_funcs import import_ecg_data

# Task 1 Plots
# Function to plot ECG signal in the time and frequency domain
def plot_ecg(name, sample_rate, ecg, beat_start, beat_end):
    n = len(ecg)
    time = np.arange(n) / sample_rate # array of times

    # Frequency domain (linear)
    ecg_fft = (1 / n) * fft(ecg)
    mag_fft = np.abs(ecg_fft)
    freqs = np.arange(n // 2) * sample_rate / n # frequency bins up to sample_rate/2

    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG Analysis: {name}")

    # fig 1: time domain
    plt.subplot(2, 1, 1)
    plt.title("Time domain")
    plt.plot(time, ecg)
    plt.xlim(0, n / sample_rate)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")

    # zoomed in
    plt.subplot(2, 1, 2)
    plt.title("Zoomed: two heartbeats")
    plt.plot(time, ecg)
    plt.xlim(beat_start, beat_end)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")

    plt.tight_layout() # stop axes overlapping

    # fig 2: frequency domain
    plt.figure(figsize=(12, 8))
    plt.suptitle(f"ECG spectrum: {name}")

    plt.subplot(2, 1, 1)
    plt.title("Full spectrum")
    plt.plot(freqs[0:], mag_fft[0:n // 2])
    plt.xlim(0, sample_rate / 2)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")

    # zoomed in
    plt.subplot(2, 1, 2)
    plt.title("Zoomed-in Spectrum")
    plt.plot(freqs[0:], mag_fft[0:n // 2])
    plt.xlim(0, sample_rate / 2)
    plt.ylim(0, 3e-5) # reveals the small peaks
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")

    plt.tight_layout() # stop axes overlapping

```

```
# import ECG Data & plot
_, ecg, sample_rate = import_ecg_data("ecg1.tsv")
plot_ecg("ECG 1", sample_rate, ecg, 5.5, 6.7) # zoom window containing two beats
plt.show()
```

C.3 fir_design.py

```
#Task 2
import numpy as np
from scipy.fft import ifft

#This function determines the FIR coefficients for freq. ranges specified in the function arguments (2D array: [low, high]) and a given sample rate.
def fir_design(sample_rate, ranges):

    ranges = np.array(ranges)
    min_width = np.min(ranges[:, 1] - ranges[:, 0])

    bin_size = min_width / 2
    M = int(sample_rate / bin_size) #half the narrowest stopband, so every stopband covers at least 2 bins

    M += M%2 # ensure M is even

    #Using folded axis to create a symmetric frequency response
    f = np.arange(M) * sample_rate / M #freq. of each bin
    f = np.minimum(f, sample_rate - f) #fold: bin M-k gets the same freq. as bin k
    H = np.ones(M) #initialise frequencies to 1 (pass band)

    #define low and high, setting the bins within this band to 0.
    for low, high in ranges:
        H[(f >= low) & (f <= high)] = 0 #remove bins in range (set to 0)

    # IFFT - produces the impulse response, keeping only the real part (H is mirrored, so complex cancels)
    h = np.real(ifft(H))

    # Swap the two halves: centres the impulse response at M/2
    h_shift = np.zeros(M)
    h_shift[0:M//2] = h[M//2:M]
    h_shift[M//2:M] = h[0:M//2]

    h_windowed = h_shift * np.hamming(M) #apply hamming window to reduce Gibbs ripple from truncation

    #Edge cases (DC and Nyquist) are covered by the folded axis (bin 0 and bin M/2 [fs/2])

    return h_windowed
```

C.4 fir_filter.py

```
# task 3a + 4a - FIRfilter class functions
import numpy as np


class FIRfilter:
    def __init__(self, _coefficients):
        """
        Parameters:
        _coefficients: array
            The M filter coefficients calculated in Task 2.
```

```
        """
        self.coefficients = np.array(_coefficients, dtype=float) # load coefficients into object
        self.ntaps = len(self.coefficients) # get buffer array length

        self.buffer = np.zeros(self.ntaps, dtype=float) # generate buffer array

        self.offset = 0 # ring buffer index where the next sample is written

    def dofilter(self, v):
        """
        Parameters:
        v: float
            The incoming input sample x[n].

        Returns:
        float
            The filtered output sample y[n].
        """
        self.buffer[self.offset] = v # load current sample into buffer

        k = self.offset + 1
        # output = sum[0:M-1](h(m) \dot x(n-m))
        output = (
            np.dot(self.coefficients[:k], self.buffer[self.offset::-1])  # fwd sum
            + np.dot(self.coefficients[k:], self.buffer[:self.offset:-1]) # backward sum
        ) # dot product multiply coefficients with samples in object

        # iterate offset position, reset to zero
        if self.offset == self.ntaps - 1:
            self.offset = 0
        else:
            self.offset += 1


        return float(output)

    def doFilterAdaptive(self, signal, noise, learningRate):
        """
        Parameters:
        signal: float
            Current contaminated ECG sample d[n].
        noise: float
            Current reference noise sample x[n] (e.g. sin(2*pi*50/fs * n)).
        learningRate: float
            LMS step size (mu) controlling how fast the weights adapt.

        Returns:
        float
            The cleaned ECG sample e[n] = d[n] - y[n].
        """
        read_idx = self.offset # store current write index before dofilter increments offset

        y = self.dofilter(noise) # filter reference noise x[n] to estimate interference y[n]

        e = signal - y # subtract estimated noise from signal to get error / clean ECG e[n]

        k = read_idx + 1
        step = learningRate * e # calculate LMS weight update factor (mu * e[n])
        # w_m[n+1] = w_m[n] + mu * e[n] * x[n-m]
        self.coefficients[:k] += step * self.buffer[read_idx::-1] # fwd tap weight update
        self.coefficients[k:] += step * self.buffer[:read_idx:-1] # backward tap weight update

        return float(e)
```

C.5 fir_realtime_sim.py

```python
# Task 3b - Simulating realtime FIR filtering importing fir_design and fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_design import fir_design
from fir_filter import FIRfilter
from support_funcs import import_ecg_data

# FIR Filter params from Task 2 (also used by heartbeat.py)
BASELINE_WANDER = [0, 0.5]  # Remove DC and low-frequency baseline drift
MAINS_AND_HF_HUM = [45, 125]  # Remove 50 Hz mains hum and high-frequency noise up to Nyquist (fs/2)

def run_fir_filter(ecg, sample_rate):
    # generate list of tap coeffs
    h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])

    # create object and generate output list
    fir = FIRfilter(h)
    ecg_filtered = []

    for sample in ecg:
        ecg_filtered.append(fir.dofilter(sample))

    # convert list into np to plot
    ecg_filtered = np.array(ecg_filtered)
    group_delay = (len(h) // 2) / sample_rate # linear-phase FIR delay: centre tap is at M/2 after the half swap
    print(f"FIR filter: M = {len(h)} taps, group delay = {group_delay:.3f} s")

    return ecg_filtered, group_delay


if __name__ == "__main__": # since its being imported we need to prevent plotting repeats
    # load ECG Data
    t, ecg, sample_rate = import_ecg_data("ecg1.tsv")

    ecg_filtered, group_delay = run_fir_filter(ecg, sample_rate)
    n = len(ecg_filtered)

    # generate plots
    # Filtered ECG (Time Domain, Zoomed Heartbeat, Frequency Spectrum) ---
    freqs = np.arange(n // 2) * sample_rate / n
    raw_fft = (1 / n) * np.abs(fft(ecg))
    filt_fft = (1 / n) * np.abs(fft(ecg_filtered))

    beat_start, beat_end = 5.5, 6.7
    in_beat = (t >= beat_start) & (t <= beat_end)

    plt.figure(figsize=(12, 8))
    plt.suptitle("Task 3: Realtime FIR Filtered ECG: ECG 1")

    # Full time domain
    plt.subplot(3, 1, 1)
    plt.title("Time Domain: Raw vs Filtered ECG")
    plt.plot(t, ecg, alpha=0.5, label="Unfiltered ECG")
    plt.plot(t, ecg_filtered, label="Filtered ECG")
    plt.xlim(0, n / sample_rate)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
```

```
    plt.legend(loc="upper right")
    plt.grid()

    # Zoomed single heartbeat (PQRST check)
    plt.subplot(3, 1, 2)
    plt.title("Zoomed Heartbeat (PQRST Complex): Raw vs Filtered ECG")
    plt.plot(t, ecg - np.mean(ecg[in_beat]), alpha=0.5, label="Unfiltered ECG (DC mean subtracted)")
    plt.plot(t - group_delay, ecg_filtered, label=f"Filtered ECG (delay-aligned by {group_delay:.1f} s)")
    plt.xlim(beat_start, beat_end)
    plt.xlabel("Time (s)")
    plt.ylabel("Amplitude (V)")
    plt.legend(loc="lower right")
    plt.grid()

    # Linear frequency spectrum
    plt.subplot(3, 1, 3)
    plt.title("Frequency Spectrum: Raw vs Filtered ECG")
    plt.plot(freqs, raw_fft[:n // 2], alpha=0.5, label="Unfiltered Spectrum")
    plt.plot(freqs, filt_fft[:n // 2], label="Filtered Spectrum")
    plt.xlim(0, sample_rate / 2)
    plt.ylim(0, 3e-5)
    plt.xlabel("Frequency (Hz)")
    plt.ylabel("Magnitude (V)")
    plt.legend(loc="upper right")
    plt.grid()

    plt.tight_layout()
    plt.show()
```

C.6 lms_filter.py

```
# Task 4b - Simulating realtime adaptive LMS filtering importing fir_filter.

import numpy as np
import matplotlib.pyplot as plt
from scipy.fft import fft

from fir_filter import FIRfilter
from fir_realtime_sim import run_fir_filter
from support_funcs import import_ecg_data

# Load ECG Data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")

# Run Task 3 FIR filter for comparison
ecg_fir, group_delay = run_fir_filter(ecg, sample_rate)

# Adaptive LMS Filter Setup: two cancellers in series

# DC / baseline wander canceller (constant reference = 1.0)
M_DC = 1 # number of taps
mu_dc = 0.0125 # learning rate
lms_dc = FIRfilter(np.zeros(M_DC)) # adaptive filter, weights start at 0

# 50 Hz mains canceller (50 Hz sine reference)
M_lms = 10 # number of taps
mu_lms = 0.01 # learning rate
f_noise = 50.0 # ref signal frequency
lms = FIRfilter(np.zeros(M_lms)) # adaptive filter, weights start at 0

# Realtime Adaptive Filtering
```

```
ecg_lms = [] # results list

for i, sample in enumerate(ecg):

    # remove DC / baseline wander
    ecg_no_dc = lms_dc.doFilterAdaptive(sample, 1.0, mu_dc)
    # remove 50 Hz powerline interference
    noise_50 = np.sin(2 * np.pi * f_noise * i / sample_rate)
    ecg_lms.append(lms.doFilterAdaptive(ecg_no_dc, noise_50, mu_lms))

ecg_lms = np.array(ecg_lms) # convert to np array to plot
n = len(ecg_lms)

# Plotting (FIR Filtered ECG vs Adaptive LMS Filtered ECG)
freqs = np.arange(n // 2) * sample_rate / n
fir_fft = (1 / n) * np.abs(fft(ecg_fir))
lms_fft = (1 / n) * np.abs(fft(ecg_lms))

beat_start, beat_end = 5.5, 6.7

plt.figure(figsize=(12, 8))
plt.suptitle("Task 4: Realtime Adaptive LMS vs FIR Filtered ECG: ECG 1")

# Full time domain
plt.subplot(3, 1, 1)
plt.title("Time Domain: FIR vs Adaptive LMS Filtered ECG")
plt.plot(t, ecg_fir, alpha=0.7, label="FIR Filtered ECG")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(0, n / sample_rate)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="upper right")
plt.grid()

# Zoomed single heartbeat (PQRST check)
plt.subplot(3, 1, 2)
plt.title("Zoomed Heartbeat (PQRST Complex): FIR vs Adaptive LMS Filtered ECG")
plt.plot(t - group_delay, ecg_fir, alpha=0.8, label=f"FIR Filtered ECG (delay-aligned by {
    group_delay:.1f} s)")
plt.plot(t, ecg_lms, alpha=0.8, label="Adaptive LMS Filtered ECG")
plt.xlim(beat_start, beat_end)
plt.xlabel("Time (s)")
plt.ylabel("Amplitude (V)")
plt.legend(loc="lower right")
plt.grid()

# Linear frequency spectrum
plt.subplot(3, 1, 3)
plt.title("Frequency Spectrum: FIR vs Adaptive LMS Filtered ECG")
plt.plot(freqs, fir_fft[:n // 2], alpha=0.7, label="FIR Spectrum")
plt.plot(freqs, lms_fft[:n // 2], alpha=0.8, label="Adaptive LMS Spectrum")
plt.xlim(0, sample_rate / 2)
plt.ylim(0, 3e-5)
plt.xlabel("Frequency (Hz)")
plt.ylabel("Magnitude (V)")
plt.legend(loc="upper right")
plt.grid()

plt.tight_layout()
plt.show()
```

C.7 heartbeat.py

```python
#5 - simulating real time heartbeat detection using fir_filter

import numpy as np
import matplotlib.pyplot as plt

from fir_design import fir_design
from fir_filter import FIRfilter
from fir_realtime_sim import BASELINE_WANDER, MAINS_AND_HF_HUM
from support_funcs import import_ecg_data


# Load and format ECG data
t, ecg, sample_rate = import_ecg_data("ecg1.tsv")  # ECG to analyse
_, template_ecg, _ = import_ecg_data("ecg2.tsv")  # other lab partner's ECG


#Apply filter from Task 3
h = fir_design(sample_rate, [BASELINE_WANDER, MAINS_AND_HF_HUM])

#Extract QRS from TEMPLATE
template_fir = FIRfilter(h)
template_clean = np.zeros(len(template_ecg))
for i in range(len(template_ecg)):
    template_clean[i] = template_fir.dofilter(template_ecg[i])


#Cut around the R peak (include QRS). R is chosen after the cleaning filter's buffer has filled
R = 2576 #sample number of R peak in template ECG (ecg2.tsv)
template = template_clean[R-12:R+13] #25 samples
matched_fir = FIRfilter(np.flip(template)) #reverse coefficients in time
ecg_fir = FIRfilter(h) #fresh cleaning filter for ECG

#matched filter output, stored for plotting
matched_fir_out = np.zeros(len(ecg))

# --- Realtime detection ---
THRESHOLD = 1.5e-11 #from matched filter plot
TRANSIENTS = len(template) + len(h) # ignore output until both filter buffers are full (cleaning
     + matched filter)
MIN_BPM = 30
MAX_BPM = 200
last_beat = None #index of previous beat
heart_rate = []
beat_idx = [] #sample index of each beat with an accepted rate

for i in range(len(ecg)):
    matched_fir_out[i] = matched_fir.dofilter(ecg_fir.dofilter(ecg[i])) ** 2 #square to make
    positive and emphasise peaks
    if i <= TRANSIENTS:
        continue
    if matched_fir_out[i] > THRESHOLD and matched_fir_out[i-1] <= THRESHOLD:
        peak = i
        if last_beat != None:
            #Ensure heartrate is within reasonable range (30-200 bpm)
            momentary_rate = 60* sample_rate / (peak - last_beat) #bpm
            in_range =  MIN_BPM < momentary_rate < MAX_BPM #Heuristic 1

            prev_rate = heart_rate[-1] if heart_rate else momentary_rate #prevent IndexError
            consistency = 0.80*prev_rate < momentary_rate < 1.20*prev_rate #Heuristic 2
            if in_range and consistency:
                heart_rate.append(momentary_rate)
                beat_idx.append(peak)
        last_beat = peak
```

```
62
63 print(f"Heart rates calculated: {len(heart_rate)}, mean heart rate: {np.mean(heart_rate):.1f}
       bpm")
64
65 # --- PLOTS ---
66 delay = (len(h) // 2 + len(template) // 2) / sample_rate # delay of cleaning + matched filter:
       centre tap of each
67 template_ms = (np.arange(len(template)) - len(template) // 2) / sample_rate * 1000 # time
       relative to R peak
68
69 plt.figure(figsize=(12, 8))
70 plt.suptitle("Matched Filter R-Peak Detection")
71
72 # Template (matched filter coefficients before time reversal)
73 plt.subplot(2, 1, 1)
74 plt.title(f"QRS Template from ECG 2 ({len(template)} samples)")
75 plt.plot(template_ms, template, marker=".")
76 plt.xlabel("Time relative to R peak (ms)")
77 plt.ylabel("Amplitude (V)")
78 plt.grid(True)
79
80 # Squared matched filter output with threshold and detected beats
81 plt.subplot(2, 1, 2)
82 plt.title("Squared Matched Filter Output: ECG 1")
83 plt.plot(t, matched_fir_out, label="Squared matched filter output")
84 plt.axhline(THRESHOLD, color="red", linestyle="--", label=f"Threshold ({THRESHOLD:.1e})")
85 plt.plot(t[beat_idx], matched_fir_out[beat_idx], "x", color="black", label=f"Detected beats ({
       len(beat_idx)})")
86 plt.axvspan(0, TRANSIENTS / sample_rate, color="grey", alpha=0.2, label="Buffer fill (ignored)")
87 plt.xlim(0, len(ecg) / sample_rate)
88 plt.ylim(0, 0.5e-10)
89 plt.xlabel(f"Time (s), output delayed by {delay:.2f} s")
90 plt.ylabel("Amplitude (V$^4$)")
91 plt.legend(loc="upper right")
92 plt.grid(True)
93
94 plt.tight_layout()
95
96 # Momentary heart rate, shifted back by the filter delay so times match the ECG
97 plt.figure(figsize=(12, 4))
98 plt.title(f"Momentary Heart Rate: ECG 1 (mean {np.mean(heart_rate):.1f} bpm)")
99 plt.plot(t[beat_idx] - delay, heart_rate, marker=".")
100 plt.xlim(0, len(ecg) / sample_rate)
101 plt.xlabel("Time (s)")
102 plt.ylabel("Heart rate (bpm)")
103 plt.grid(True)
104
105 plt.tight_layout()
106
107 plt.show()
```